

Quant Developer Assignment

Sequential development plan · Vera Developers Pvt. Ltd. · Broker: Angel One SmartAPI

Submission due	1 October 2026
Broker	Angel One SmartAPI (REST + WebSocket 2.0). HR approved it in place of Zerodha Kite Connect, which the email names.
Tooling	Developed primarily with Claude Code, as the email asks.
Orders	Paper broker only. No real orders: live API orders need a registered static IP and would risk real money.
Market data	Real SmartAPI data (historical candles and live WebSocket) plus a synthetic generator for tests.

How to use this plan

- Do the steps strictly in order. Each step only uses what earlier steps produced; the **Builds on** line shows which.
- Do not start a step until the previous step's **Done when** check passes. Commit after every step.
- Step 11 is a checkpoint: everything up to it is a complete, tested system on its own. Push it before moving on.
- If time runs short anywhere after step 11, jump straight to step 17 and list the unbuilt steps in the README as designed but not built.
- Model: Claude Opus 5.5 in Claude Code. Each step lists a suggested effort level (Opus 5.5 defaults to medium). Start with `claude --model opus --effort high`; change mid-session with `/effort`.

The plan: 17 sequential steps

Phase 1 · Setup

1	Repo and conventions	effort: medium
Builds on	Nothing (first step)	
Build	• Poetry project with a <code>src/</code> layout, pandas, numpy, pytest, ruff, mypy and pre-commit. • Structured JSON logging set up once, used by every module. • README skeleton that states scope and assumptions up front (paper trading, SmartAPI approved by HR, real + synthetic data). • Git: small, meaningful commits (e.g. conventional-commit messages). • Send Purnima a one-line email confirming that SmartAPI is approved in place of Kite Connect, so it is in writing.	
Done when	An empty test suite and all linters pass.	
Covers (email)	Must-haves: Python tooling (pandas, numpy, virtualenv/poetry, pytest), Git.	

2	SDLC automation with Claude Code	effort: high
Builds on	Step 1	
Build	• CLAUDE.md with the hard rules: money is never a float; no lookahead; backtest and live share one code path; the broker is only used through an interface; SmartAPI rate limits are always respected; every strategy change starts with a failing regression test. • Subagents in <code>.claude/agents/</code>: a <i>test-writer</i> (writes the failing regression test first) and a <i>reviewer</i> (checks for lookahead, float money, missing idempotency, rate-limit bypass, missing tests). • Hooks that run lint and tests automatically after edits. • CI (GitHub Actions) running the full suite on every push. • Use this loop for every step after this one.	
Done when	A small change goes test-writer → implement → reviewer → green CI.	
Covers (email)	Develop with Claude Code; Make agents to automate the SDLC; Write tests; Git code review.	

Phase 2 · Market plumbing

3

Domain model and money rules

effort:
medium

Builds on Step 1, 2

Build

- Typed dataclasses: Instrument, Bar, Tick, OrderIntent, Order, Fill, Position.
- Money as Decimal or integer paise; prices snapped to tick size; one rounding policy in one place.

Done when Rounding and arithmetic tests pass; mypy is clean.

Covers (email) Must-haves: dataclasses, type hints, numerical discipline ("matches to the paisa").

4

Contract masters, expiry and rollover

effort: high

Builds on Step 3

Build

- Load Angel One's scrip master (a saved snapshot as a test fixture; refreshed daily in live mode).
- Key instruments on exchange + trading symbol, keeping the symbol token for API calls (tokens can be reused across expiries).
- Check the units of the strike and tick-size fields against exchange specs before trusting them.
- MCX spec table: lot size, tick size, price quotation basis, expiry, staggered-delivery window. NSE F&O basics: lot size, expiry.
- P&L multiplier from quotation basis, e.g. Gold is quoted per 10 g but traded in 1 kg lots, so ₹1 on the quote = ₹100 per lot.
- Roll rule: N days before expiry, or before the delivery window on MCX. Continuous-contract mapping.

Done when Multiplier, expiry and roll-date tests pass.

Covers (email) MCX and NSE contract masters; expiry and rollover; MCX contract specs.

5

Cost model

effort: high

Builds on Step 3, 4

Build

- Angel One brokerage, STT (NSE F&O), CTT (MCX), exchange transaction charges, SEBI fee, stamp duty and GST, per leg.
- Reference sheet of ~10 trades across NSE F&O and MCX from Angel One's brokerage calculator, plus a few of your own real contract notes (the strongest proof).

Done when The engine matches the reference sheet and contract notes to the paisa.

Covers (email) STT/CTT and brokerage mechanics; "matches to the paisa".

Phase 3 · Data and signals

6	Market data layer	effort: medium
Builds on	Step 3, 4	
Build	• Bar and tick schemas; Parquet or DuckDB storage. • Historical download of real NSE F&O and MCX bars via getCandleData, throttled to 3/sec, 150/min, 5,000/hr. • Synthetic generator (trends, chop, gaps, volatility spikes) for tests. • Tick-to-bar aggregator for live; continuous futures series stitched with step 4's roll rule. • A TickSource interface so SmartAPI WebSocket, recorded replays or a vendor like TrueData/GDFL plug in the same way. • Raw tick recorder that writes ticks to Parquet and replays them through the same TickSource interface (a tick-level pipeline, like TrueData/GDFL-style feeds).	
Done when	Real and synthetic data feed one bar iterator; aggregator tests pass.	
Covers (email)	Tick vendors; data integration; storage (good-to-have).	
7	Technical analysis module	effort: high
Builds on	Step 6	
Build	• One incremental (update-per-bar) implementation per indicator, built on shared primitives (EMA, true range, rolling window) so no formula is written twice. • Trend: EMA, ADX. Momentum: RSI, MACD. Volatility: ATR, Bollinger. Volume: OBV, VWAP. • The same objects run bar by bar in backtest and live, so the two cannot drift apart. No iterrows() in hot loops. • Tests compare against TA-Lib or pandas-ta (test-only dependency). • Use pandas/numpy vectorisation where it is the right tool (loading, resampling, batch statistics, reports) and incremental per-bar objects in the live and backtest hot path. Write a short README note on why each is used where (the email asks when vectorisation is the wrong answer).	
Done when	Every indicator matches the reference after warm-up.	
Covers (email)	Technical analysis module; the iterrows / vectorisation must-have.	

Phase 4 · Trading logic

8	Strategy engines	effort: high
Builds on	Step 7	
Build	• ATR grid: levels at $k \times \text{ATR}$ around an anchor, pyramiding up to N units, exits per level. • Stop-and-reverse: an ATR trailing stop that flips the position when hit. • Both are pure functions of (bar, indicators, position, params) returning order intents. • Params are a dataclass so the regime engine can override them in step 15.	
Done when	Scenario tests pass for a trend, a choppy market and a gap through several grid levels.	
Covers (email)	Grid and stop-and-reverse engines; ATR spacing; pyramiding.	
9	Risk layer	effort: high
Builds on	Step 8	
Build	• Every intent passes pre-trade checks: per-instrument and total position caps, max order size. • Kill switches: manual, max daily loss, max drawdown, reject storm, stale data. They block new orders and can flatten.	
Done when	Tests show caps clipping orders and each kill switch halting trading.	
Covers (email)	Kill switches; position caps.	
10	Order and state management	effort: xhigh
Builds on	Step 3, 9	
Build	• Order state machine. • Deterministic idempotency key hashed into SmartAPI's ordertag. Confirm the max length in the docs (the SmartAPI forum reports 15 characters) and size the hash to fit. • SQLite write-ahead journal: each intent is recorded before it is sent. • Positions and P&L derived only from fills. State rebuilt from the journal on start.	
Done when	Duplicate submissions are no-ops; a crash mid-order recovers cleanly; P&L from fills matches expected.	
Covers (email)	Idempotent placement; position and P&L truth; crash recovery.	

Phase 5 · Backtest (checkpoint)

11	Paper broker and backtest harness	effort: xhigh
Builds on	Step 5, 6, 7, 8, 9, 10	
Build	• PaperBroker behind the same broker interface the SmartAPI adapter will use. • Fill rules: decide at bar close; market orders fill at the next open; stops/limits fill when the next bar crosses them (at the open if it gaps through); if both sides trigger in one bar, assume the adverse one first. • Slippage in ticks; costs from step 5. • Loop: data → TA → strategy → risk → orders → broker, writing a trade blotter. • Walk-forward runner (rolling in-sample fit, out-of-sample evaluation). • No-lookahead test: a run on data cut off at time t makes identical decisions up to t. • Golden-file regression test on a fixed dataset. • Optional cross-check: run one simple strategy (e.g. an EMA cross) in vectorbt or backtrader and compare its trades with your harness; note every difference (fill timing, costs, MCX multipliers). This is the evidence for the tool-opinion note in step 17.	
Done when	A full backtest runs end to end and all three tests pass. Checkpoint: push it.	
Covers (email)	Backtest harness: bar-accurate fills, slippage and cost modelling, walk-forward, no lookahead.	

Phase 6 · Live integration (Angel One SmartAPI)

12	SmartAPI REST adapter	effort: high
Builds on	Step 10, 11	
Build	• SmartAPI broker class on the official SmartAPI Python SDK, implementing the same interface as the PaperBroker; tested against a mocked client. • Login with client code + PIN + TOTP; session refresh with generateTokens (1/sec, 1,000/hr); re-login if the token is invalid. • Client-side rate limiter built from the docs table (Appendix A): place + modify + cancel combined 9/sec; orders 500/min and 1,000/hr; order book, positions and trade book 1/sec; order status and quotes 10/sec. • On SmartAPI a 403 means the rate limit was hit, not an expired session: back off, don't re-login. • Retries with exponential backoff, but never blindly re-send an order: look it up by ordertag in the order book first. • Pre-trade margin check via the margin calculator API (10/sec), fed into step 9's risk layer.	
Done when	Mocked tests cover a timeout followed by a duplicate check, a burst of 403s and an expired session.	
Covers (email)	Broker integration (REST); auth/token refresh; rate limits; retries with backoff; idempotency; margins.	

13	Live market data feed	effort: xhigh
Builds on	Step 6, 12	
Build	• asyncio consumer for SmartAPI WebSocket 2.0 (limits: 3 connections per client code, 1,000 token subscriptions per session). • Bounded queue with an explicit back-pressure policy: conflate ticks per instrument, never drop order updates. Confirm in the docs whether order updates arrive on a separate order-status stream (then run two consumers). • The SDK calls back from its own thread: hand ticks to your event loop with <code>call_soon_threadsafe</code>. • Heartbeat as the docs specify; treat silence as a dead connection. • Reconnect-and-resync: resubscribe, backfill missed bars via <code>getCandleData</code>, re-check orders. • Graceful shutdown: stop new intents, flush the journal, close sockets cleanly. • Race-condition stress test: hammer ticks, order updates and shutdown concurrently and assert identical state on every run. If it finds a real race, fix it and record it in the README.	
Done when	A fake feed that disconnects mid-stream produces the same bars as an uninterrupted run.	
Covers (email)	WebSocket integration; concurrency (queues, back-pressure, graceful shutdown); reconnect-and-resync.	

14	Live runner and restart reconciliation	effort: xhigh
Builds on	Step 11, 12, 13	
Build	• Assemble the live engine from the same components; broker chosen by config (paper by default). • Live SmartAPI data + paper broker = a real live test without real orders. • On startup: load the journal, fetch orders, trades and positions (respecting their 1/sec limits), diff, adopt or flag mismatches, then resume. • Parity test: replaying recorded live bars through the backtest gives identical fills and P&L.	
Done when	A kill-and-restart mid-session ends with state identical to the broker's.	
Covers (email)	Order-state reconciliation after restarts; backtest numbers reconcile to live numbers.	

Phase 7 · Regime, observability, submission

15	Macro Regime Engine	effort: high
Builds on	Step 8, 9, 11	
Build	• Ingest daily macro proxies (e.g. India VIX, USDINR, crude, bond yields) from CSV, time-stamped to when each value was actually known. • Score them (z-scores or percentiles) and map to regime states with hysteresis so they don't flap. • Each state overrides step 8's params (grid spacing, pyramid depth, caps). • Circuit breakers wired into step 9's risk layer, e.g. a VIX spike halts new entries.	
Done when	Tests cover state changes, overrides reaching the strategy and a breaker tripping; the backtest runs with the regime on and off.	
Covers (email)	Macro Regime Engine: ingest, score, override, circuit breakers.	

16	Observability and reconciliation	effort: high
Builds on	Step 11, 14	
Build	• JSON logs carrying the order tag as a correlation ID. • Trade blotter in the desk's spreadsheet layout. • Phone alerts (Telegram bot or ntfy) on position mismatch, P&L drift, kill-switch or breaker trips, stale feed and rejects. • Reconciliation report comparing backtest, live/paper and a hand-built spreadsheet to the paisa. • FastAPI status service (health, positions, P&L, regime, kill-switch state) with a Streamlit dashboard on top (blotter, open orders, P&L curve). • Grafana monitoring (good-to-have): export metrics from the status service and chart them, run via docker-compose.	
Done when	A forced mismatch pings your phone; the report shows zero difference on the reference trades.	
Covers (email)	Observability; backtest ↔ live ↔ desk spreadsheet reconciliation; Streamlit / FastAPI / Grafana (good-to-have).	

17	Package and submit	effort: medium
Builds on	Step 1-16	
Build	• README: architecture diagram, how to run, scope and assumptions (including HR's approval of SmartAPI in place of Kite Connect), what is live-ready vs paper-only and why, and a table mapping every line of the email to modules and tests (Appendix B). • Dockerfile and docker-compose (app, plus optional Redis for a latest-tick cache and Grafana). Postgres/TimescaleDB can replace Parquet/DuckDB if you want a SQL time-series store (all good-to-have). • README section "Tool opinions": why a custom harness rather than vectorbt or backtrader, and the limits of TA-Lib and pandas-ta, written from what you saw in steps 7 and 11. • In the reply email and README, mention any live trading you have done with your own or someone else's capital (good-to-have), e.g. your TrendPulse platform. • Final full test run, tidy commit history, push, reply to Purnima with the repo link.	
Done when	A fresh clone runs the tests and a demo backtest with one command.	
Covers (email)	Submission; Git; Docker, SQL / time-series, Redis, live-trading experience, tool opinions (good-to-have).	

Appendix A · SmartAPI rate limits the engine must respect

From Angel One's SmartAPI Rate Limit page. Limits are counted per client code. Exceeding a limit returns **403 Access denied because of exceeding rate limit**. Place, modify and cancel order requests are counted together and must not exceed **9 per second combined**.

Endpoint	Used in step	Per second	Per minute	Per hour
loginByPassword	12	1	NA	NA
generateTokens (session refresh)	12	1	NA	1,000
getProfile	12	3	NA	1,000
getRMS (funds)	12	2	NA	NA
placeOrder / modifyOrder / cancelOrder	12	9 combined	500	1,000
getOrderBook	12, 14	1	NA	NA
details/{GuiOrderID} (single order status)	12, 14	10	500	5,000
getTradeBook	14	1	NA	NA
getPosition	14	1	NA	NA
getLtpData	13	10	500	5,000
market/v1/quote	13	10	500	5,000
margin/v1/batch (margin calculator)	12	10	500	5,000
historical getCandleData	6, 13	3	150	5,000
searchScrip	4	1	NA	NA
GTT create / modify / cancel	(not used)	9	500	5,000

WebSocket 2.0: 3 concurrent connections per client code; 1,000 token subscriptions per session (each token-and-mode pair counts separately).

Sources: <https://smartapi.angelbroking.com/docs/RateLimit> and the WebSocket 2.0 section of the same docs. Items marked "confirm in the docs" in the steps (ordertag length, order-update stream) should be verified there before coding.

Appendix B · Traceability: every line of the email → steps

Email requirement	Steps
Live grid and stop-and-reverse engines: ATR spacing, pyramiding	8
Kill switches, position caps	9
Technical analysis module, single tested implementation each, no duplicated math	7
Macro Regime Engine: proxies, regime states, parameter overrides, circuit breakers	15
Broker and data integration (SmartAPI in place of Kite Connect, REST + WebSocket)	12, 13
Tick vendors	6, 13
MCX and NSE contract masters, expiry and rollover	4
Backtest harness: bar-accurate fills, slippage and costs, walk-forward, no lookahead	5, 11
Backtest ↔ live ↔ desk spreadsheet reconciliation	14, 16
Idempotent placement, order-state reconciliation after restarts	10, 12, 14

Email requirement	Steps
Position and P&L truth, crash recovery	10, 14
Structured logs, trade blotters, phone alerts on deviation	1, 16
Write tests: every strategy change ships with a regression test	2 + every step's Done when
Make agents to automate the SDLC; develop primarily with Claude Code	2
Must-have: Python (pandas, numpy, dataclasses, type hints, poetry, pytest, iterrows)	1, 3, 7
Must-have: concurrency (WebSocket consumers, queues, back-pressure, graceful shutdown)	13
Must-have: API integration (auth/token refresh, rate limits, retries, idempotency, resync)	10, 12, 13
Must-have: Indian market plumbing (MCX specs, NSE F&O, margins, STT/CTT, brokerage)	4, 5, 12
Must-have: numerical discipline (matches to the paisa)	3, 5, 16
Must-have: Git (meaningful commits, code review)	1, 2, 17
Must-have: knowing when vectorisation is the wrong answer	7
Must-have: having debugged a race condition you caused (stress test and README note)	13
Good-to-have: live trading experience with own or someone else's capital	17
Good-to-have: TA-Lib / pandas-ta, vectorbt or backtrader, and an opinion on their limits	7, 11, 17
Good-to-have: Postgres / TimescaleDB / DuckDB / Parquet, Redis, Docker	6, 17
Good-to-have: Streamlit / FastAPI dashboards; Grafana monitoring	16
Good-to-have: tick-level data pipelines (TrueData, GDFL or similar)	6, 13
Deadline: submit by 1 October 2026	17